

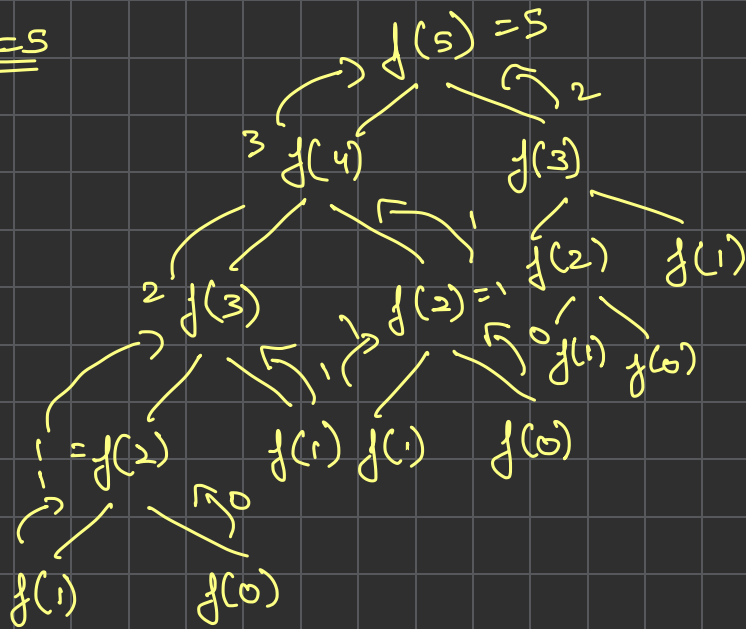
# RECURSION & TREES

⇒ A programming & mathematical technique where a function calls itself or indirectly to solve a problem by breaking it down into smaller, similar subproblems.

Example: fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

index: 0 1 2 3 4 5 6 7 8  
          ↑ ↑ ↑ =

n=5



$f(n)$

{

if ( $n == 0$ ) return 0;

if ( $n == 1$ ) return 1;

return  $f(n-1) + f(n-2)$ ;

}

# PROBLEM - 1

## 78. Subsets

Medium Topics Companies

Given an integer array `nums` of **unique** elements, return *all possible subsets* (the power set).

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

**Input:** `nums = [1,2,3]`

**Output:** `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

Example 2:

**Input:** `nums = [0]`

**Output:** `[[], [0]]`

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- All the numbers of `nums` are **unique**.

A **subset** of an array is any selection of elements from the original array, where the elements do not need to be contiguous (next to each other) and their relative order does not have to be preserved

`nums = [1, 2, 3]`

`[]`

`[1], [2], [3]`

`[1,2], [1,3], [2,3]`

`[1,2,3]`

pick/not-pick technique.

`nums = [1, 2, 3]`

|              |   |   |   |                      |
|--------------|---|---|---|----------------------|
| $\downarrow$ |   |   |   |                      |
| 0            | 1 | 2 | 3 |                      |
| x            | x | x |   | <code>[]</code>      |
| ✓            | x | x |   | <code>[1]</code>     |
| x            | ✓ | x |   | <code>[2]</code>     |
| x            | x | ✓ |   | <code>[3]</code>     |
| ✓            | ✓ | x |   | <code>[1,2]</code>   |
| ✓            | x | ✓ |   | <code>[1,3]</code>   |
| x            | ✓ | ✓ |   | <code>[2,3]</code>   |
| ✓            | ✓ | ✓ |   | <code>[1,2,3]</code> |

`result = []`

```
function(index, ds)
{
    if (index > (n-1))
    {
        result.append(ds)
        return
    }
    # pick
    ds.append(nums[index])
    function(index+1, ds)
    ds.pop()

    # not pick
    function(index+1, ds)
}
```

```

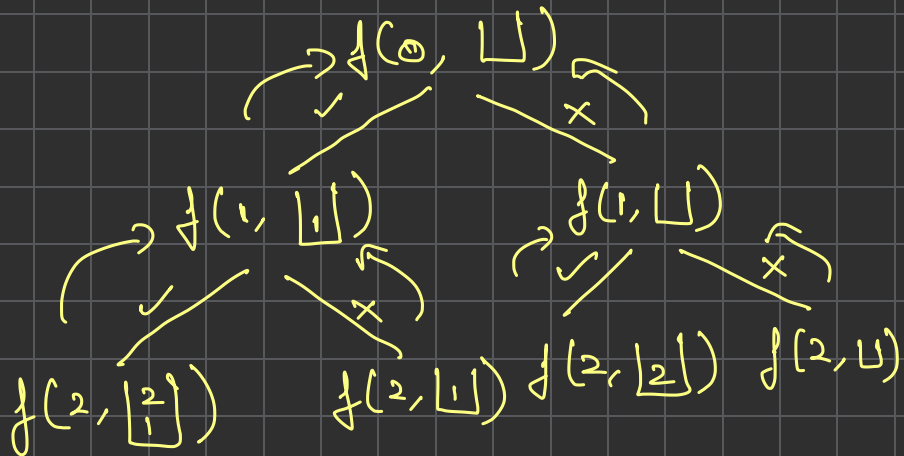
f(index, ds)
{
    if (index > (n-1)) ✓
    {
        result.append(ds)
        return
    }

    # pick
    ds.append(nums[index])
    f(index+1, ds)
    ds.pop()

    # not pick
    f(index+1, ds)
}

```

$nums = [1, 2]$   
 $result = [[1, 2], [1], [2], []]$



Time complexity =  $2^n \times n$   
 S.C =  $O(n)$

$[1, 2, \dots, n]$   
 $2 \quad \quad \quad 2$

# PROBLEM-2

## 39. Combination Sum

Medium Topics Companies

Given an array of **distinct** integers `candidates` and a target integer `target`, return a list of all **unique combinations** of `candidates` where the chosen numbers sum to `target`. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the **frequency** of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than 150 combinations for the given input.

### Example 1:

**Input:** `candidates = [2,3,6,7]`, `target = 7`

**Output:** `[[2,2,3],[7]]`

#### Explanation:

2 and 3 are candidates, and  $2 + 2 + 3 = 7$ . Note that 2 can be used multiple times. 7 is a candidate, and  $7 = 7$ .

These are the only two combinations.

### Example 2:

**Input:** `candidates = [2,3,5]`, `target = 8`

**Output:** `[[2,2,2,2],[2,3,3],[3,5]]`

### Example 3:

**Input:** `candidates = [2]`, `target = 1`

**Output:** `[]`

arr =  $\begin{bmatrix} 0 \\ 2, 3, 6, 7 \end{bmatrix}$

target = 7

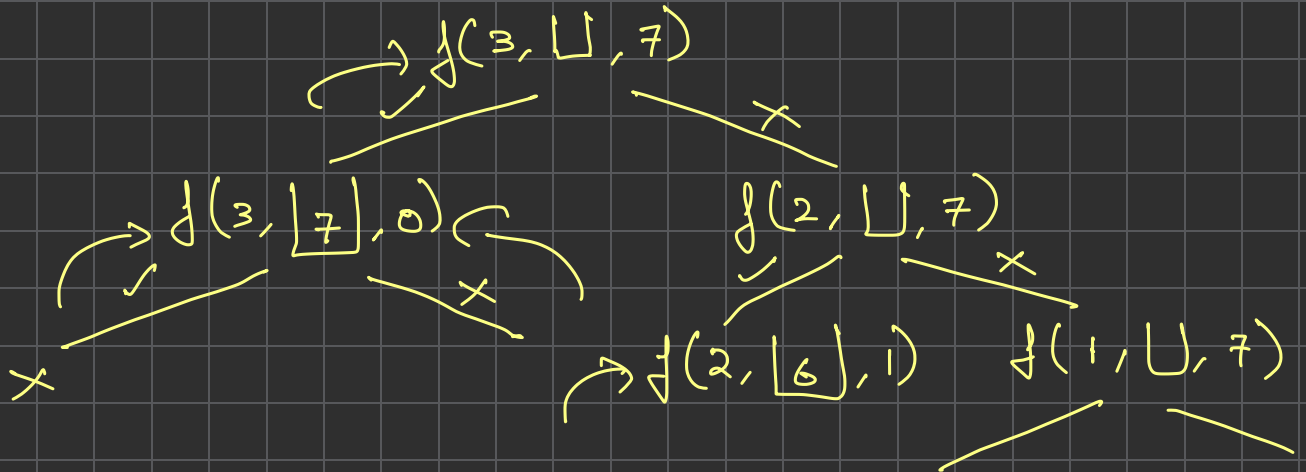
$[7], [2, 2, 3]$

result =  $[\ ]$

f(index, ds, target)

arr =  $\begin{bmatrix} 0 \\ 2, 3, 6, 7 \end{bmatrix}$ , target = 7

res =  $[[7],$



result =  $[\ ]$

f(index, ds, target)

{ # base case

if (index == -1)

if (target == 0)

result.append(ds)

return

# pick

f(index, ds + [nums[index]],

target - nums[index])

if (index == -1)

if (target == 0)

result.append(ds)

return

TC =  $2^t \cdot k$

SC =  $O(t)$

# not pick

f(index - 1, ds, target)

}

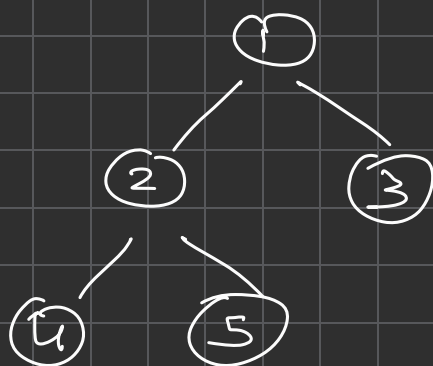
# Trees:

DFS & BFS / level order

T.C =  $O(n)$

S.C =  $O(n)$

- 1.) Pre-order (root, left, right)
- 2.) In-order (left, root, right)
- 3.) post-order (left, right, root)



pre-order: [1, 2, 4, 5, 3]

in-order: [4, 2, 5, 1, 3]

post-order: [4, 5, 2, 3, 1]

Dfs:

T.C =  $O(n)$

S.C =  $O(n)$

recursive:

pre-order = dfs(root)

{

if (root == None):

return

print(root.val)

dfs(root.left)

dfs(root.right)

}

Bfs:

T.C =  $O(n)$

S.C =  $O(n)$



## 104. Maximum Depth of Binary Tree

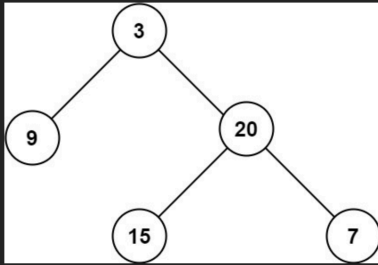
Easy Topics Companies

Given the `root` of a binary tree, return its maximum depth.

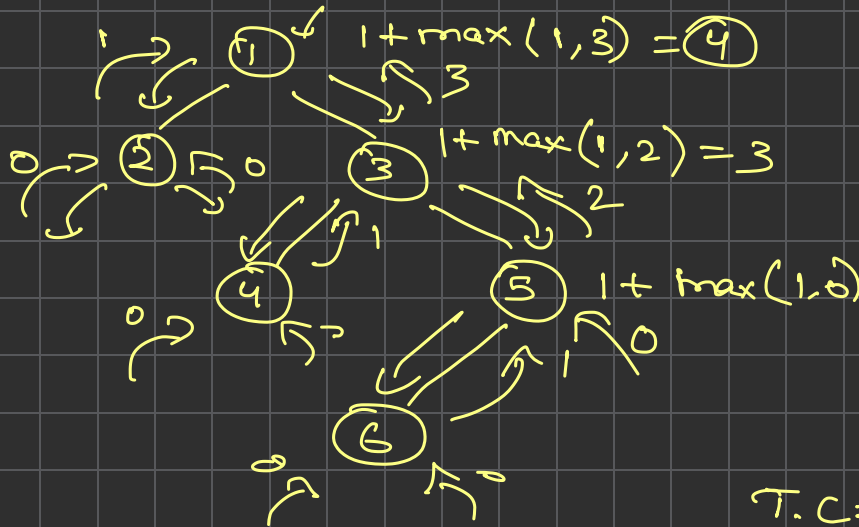
height

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

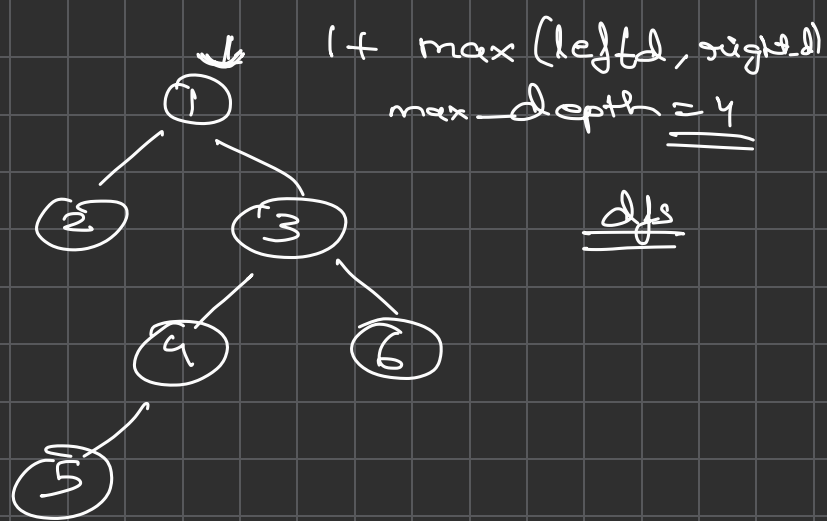
Example 1:



Input: root = [3,9,20,null,null,15,7]  
Output: 3



T.C =  $O(n)$   
S.C =  $O(n)$



```

function f(root)
{
    if (root == None)
        return 0

    ld = f(root.left)
    rd = f(root.right)
    return 1 + max(ld, rd)
}
  
```

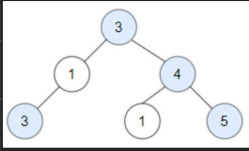
## 1448. Count Good Nodes in Binary Tree

Medium Topics Companies Hint

Given a binary tree `root`, a node `X` in the tree is named **good** if in the path from `root` to `X` there are no nodes with a value greater than `X`.

Return the number of **good** nodes in the binary tree.

Example 1:



**Input:** `root = [3,1,4,3,null,1,5]`

**Output:** 4

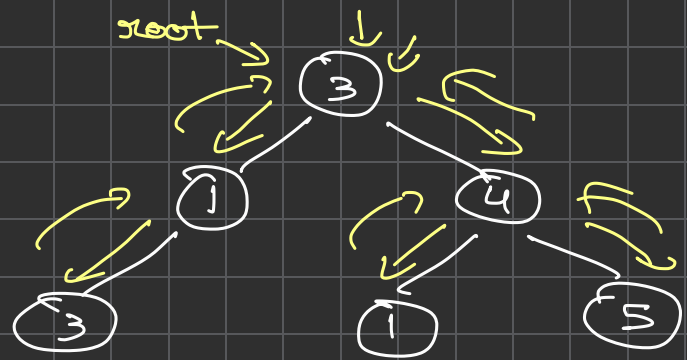
**Explanation:** Nodes in blue are **good**.

Root Node (3) is always a good node.

Node 4 → (3,4) is the maximum value in the path starting from the root.

Node 5 → (3,4,5) is the maximum value in the path

Node 3 → (3,1,3) is the maximum value in the path.



max-so-far = ~~3~~ 4 5  
good-nodes = ~~3~~ 1 ~~3~~ 4

pseudocode:

```

✓ dfs(root, max-so-far)
{
  if (root == None)
    return 0

```

ng = 0

if root.val >= max-so-far:

ng += 1

max-so-far = root.val

return ng + dfs(root.left) + dfs(root.right)

}

T.C = O(n)  
 S.C = O(n)